

IPC Through Message Passing and Message Queues

Table of Contents

1. [Introduction](#)
 2. [What is Message Passing?](#)
 3. [Basic Operations: send and receive](#)
 4. [Design Choices in Message Passing](#)
 5. [Message Queues](#)
 6. [System V Message Queue API](#)
 7. [Working Code Example](#)
 8. [POSIX Message Queues](#)
 9. [Useful Commands](#)
 10. [Advantages and Disadvantages](#)
 11. [Real-World Use Cases](#)
 12. [Summary](#)
 13. [References](#)
-

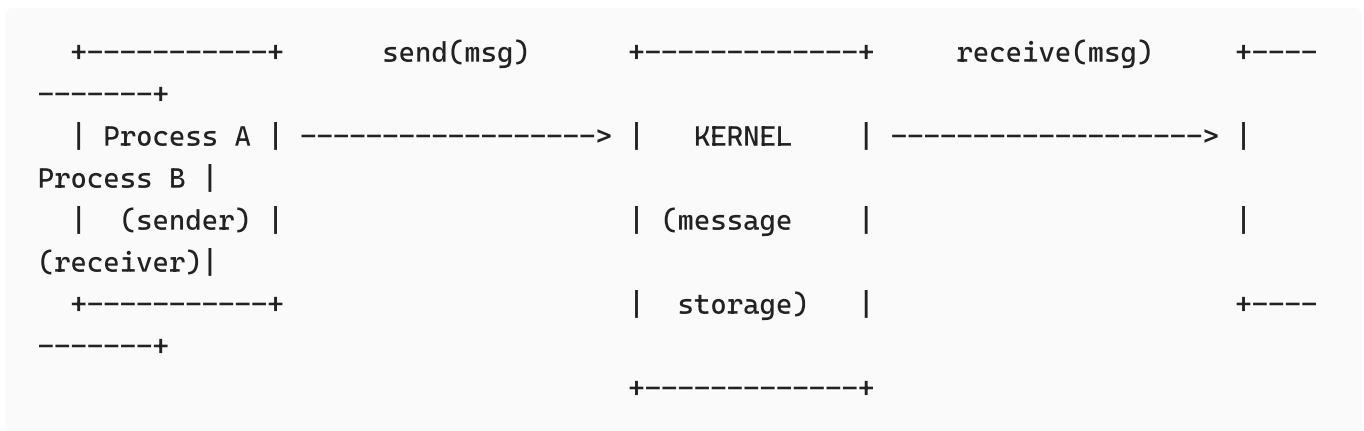
1. Introduction

Processes are isolated from each other, so they need a way to communicate. The two main models are **shared memory** and **message passing**. This document covers **message passing**, where processes communicate by **sending and receiving messages** through the operating system kernel.

Analogy: Shared memory is like two people writing on the same whiteboard. Message passing is like two people sending letters through a post office. Nobody touches the other's space, and the post office (the kernel) handles delivery.

2. What is Message Passing?

In message passing, one process **sends** a message and another process **receives** it. The kernel sits in the middle, copying the message from the sender into kernel memory and later into the receiver.



Why use it?

- Simpler and safer than shared memory, because the kernel handles most of the coordination.
- No need for the programmer to worry about two processes touching the same memory at the same time.
- Works well for **small amounts of data**.
- The same idea extends to **distributed systems** (processes on different computers).

3. Basic Operations: send and receive

Every message passing system provides at least two operations:

Operation	Meaning
<code>send(destination, message)</code>	Send a message to a process or queue
<code>receive(source, message)</code>	Get a message from a process or queue

For two processes to communicate they need a **communication link** between them. How this link is set up gives us the design choices below.

4. Design Choices in Message Passing

4.1 Naming: Direct vs. Indirect Communication

Direct communication: the processes name each other explicitly.

```
send(P, message)    // send a message to process P
receive(Q, message) // receive a message from process Q
```

- A link is created automatically between exactly one pair of processes.
- Simple, but if a process changes its identity, all references must be updated.

Indirect communication: messages go to and come from a **mailbox** (also called a **port** or **queue**).

```
send(A, message)    // send a message to mailbox A
receive(A, message) // receive a message from mailbox A
```

```
+-----+
| Process P | --send--> [ MAILBOX A ] --receive--> | Process Q |
+-----+                [msg][msg][msg]          +-----+
                        ^
                        +---receive--- Process R (can also use it)
```

- Many processes can share one mailbox.
- More flexible: sender and receiver do not need to know each other.
- **Message queues in Linux use this model.**

Feature	Direct	Indirect
Processes must know each other	Yes	No
Link between	Exactly two processes	Many processes possible
Flexibility	Low	High

4.2 Synchronization: Blocking vs. Non-blocking

Type	Sender behaviour	Receiver behaviour
Blocking (synchronous)	Waits until the message is received	Waits until a message arrives
Non-blocking (asynchronous)	Sends and continues immediately	Gets a message if available, otherwise gets nothing and continues

If both sender and receiver are blocking, this is called a **rendezvous**: both must be ready at the same time.

4.3 Buffering

The link between processes holds messages in a temporary queue. Its capacity decides the behaviour:

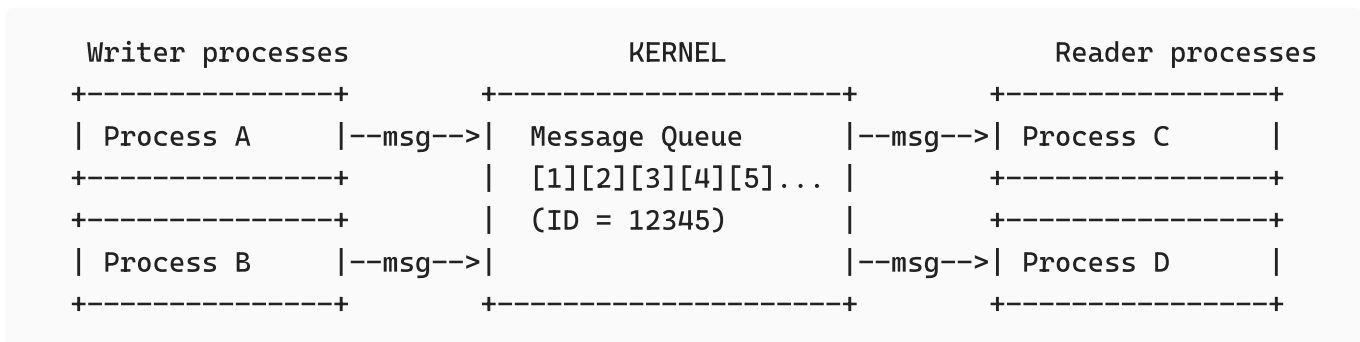
Capacity	Meaning
Zero capacity	No buffer. Sender must wait for the receiver (rendezvous).
Bounded capacity	Fixed size queue. Sender waits if the queue is full.
Unbounded capacity	Infinite queue in theory. Sender never waits.

4.4 Message Size

- **Fixed-size messages:** easy for the system to implement, harder for programmers.
- **Variable-size messages:** harder for the system, easier for programmers.

5. Message Queues

A **message queue** is a linked list of messages stored **inside the kernel**. Processes add messages to it and remove messages from it. It follows the indirect communication model.



Key features:

- Messages are stored **until they are read**, even if the sender has already exited.
- Each message has a **type** (a positive number), so a receiver can pick specific messages instead of always taking the first one.
- The queue is identified by a **key** and an **ID**.
- The queue is **persistent** until it is explicitly removed or the system reboots.

6. System V Message Queue API

Header files: `<sys/types.h>` , `<sys/ipc.h>` , `<sys/msg.h>`

6.1 The Message Structure

Every message must start with a `long` type field:

```
struct msg_buffer {  
    long mtype;           // message type (must be > 0)  
    char mtext[100];      // message data (can be any structure)  
};
```

6.2 The Four Key Functions

Function	Purpose
<code>ftok(path, id)</code>	Generate a unique key from a file path and a number
<code>msgget(key, flags)</code>	Create a new queue or get an existing one. Returns the queue ID .
<code>msgsnd(msqid, &msg, size, flags)</code>	Send a message to the queue
<code>msgrcv(msqid, &msg, size, type, flags)</code>	Receive a message from the queue
<code>msgctl(msqid, cmd, buf)</code>	Control the queue, for example remove it with <code>IPC_RMID</code>

6.3 Function Details

`msgget`

```
int msgid = msgget(key, 0666 | IPC_CREAT);
```

- `0666` gives read and write permission (same style as file permissions).
- `IPC_CREAT` creates the queue if it does not exist.
- Returns `-1` on error.

`msgsnd`

```
msgsnd(msgid, &message, sizeof(message.mtext), 0);
```

- The size is the size of the data **excluding** the `mtype` field.

- The last argument is flags. `0` means block if the queue is full. `IPC_NOWAIT` returns immediately with an error instead.

`msgrcv`

```
msgrcv(msgid, &message, sizeof(message.mtext), 1, 0);
```

The fourth argument (`msgtyp`) decides which message is picked:

msgtyp value	Behaviour
<code>0</code>	Read the first message in the queue, whatever its type
<code>> 0</code>	Read the first message of exactly that type
<code>< 0</code>	Read the message with the lowest type that is less than or equal to <code>abs(msgtyp)</code>

`msgctl`

```
msgctl(msgid, IPC_RMID, NULL); // delete the queue
```

7. Working Code Example

Two separate programs: a **sender** and a **receiver**. They communicate through a message queue.

7.1 Sender

File: `code/message_queue/sender.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>

struct msg_buffer {
    long mtype;
    char mtext[100];
};
```

```

int main(void) {
    // Generate a unique key (both programs must use the same path and id)
    key_t key = ftok("/tmp", 65);
    if (key == -1) {
        perror("ftok");
        exit(1);
    }

    // Create the queue (or open it if it already exists)
    int msgid = msgget(key, 0666 | IPC_CREAT);
    if (msgid == -1) {
        perror("msgget");
        exit(1);
    }

    struct msg_buffer message;
    message.mtype = 1; // message type 1
    printf("Enter a message to send: ");
    fgets(message.mtext, sizeof(message.mtext), stdin);
    message.mtext[strcspn(message.mtext, "\n")] = '\0'; // remove newline

    if (msgsnd(msgid, &message, sizeof(message.mtext), 0) == -1) {
        perror("msgsnd");
        exit(1);
    }

    printf("Message sent: %s\n", message.mtext);
    return 0;
}

```

7.2 Receiver

File: code/message_queue/receiver.c

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/msg.h>

struct msg_buffer {
    long mtype;
    char mtext[100];
};

```

```

int main(void) {
    key_t key = ftok("/tmp", 65);           // same key as the sender
    if (key == -1) {
        perror("ftok");
        exit(1);
    }

    int msgid = msgget(key, 0666 | IPC_CREAT);
    if (msgid == -1) {
        perror("msgget");
        exit(1);
    }

    struct msg_buffer message;

    // Receive a message of type 1 (blocks until one arrives)
    if (msgrcv(msgid, &message, sizeof(message.mtext), 1, 0) == -1) {
        perror("msgrcv");
        exit(1);
    }

    printf("Message received: %s\n", message.mtext);

    // Delete the queue
    msgctl(msgid, IPC_RMID, NULL);
    return 0;
}

```

7.3 Compile and Run

```

gcc sender.c -o sender
gcc receiver.c -o receiver

```

```

./sender      # Terminal 1
./receiver    # Terminal 2

```

Sample output:

Terminal 1 (sender):

```

Enter a message to send: Hello from the sender!
Message sent: Hello from the sender!

```

Terminal 2 (receiver):

Message received: Hello from the sender!

Try this: run `./sender` **three times** before running `./receiver`. The messages wait in the kernel. Then run `./receiver`. It picks up the first message and deletes the queue. Check what happens to the others.

8. POSIX Message Queues

POSIX offers a newer, cleaner API for message queues.

Feature	System V	POSIX
Header	<code><sys/msg.h></code>	<code><mqqueue.h></code>
Create/open	<code>msgget()</code>	<code>mq_open()</code>
Send	<code>msgsnd()</code>	<code>mq_send()</code>
Receive	<code>msgrcv()</code>	<code>mq_receive()</code>
Close/remove	<code>msgctl(IPC_RMID)</code>	<code>mq_close()</code> and <code>mq_unlink()</code>
Identified by	Numeric key	Name like <code>/myqueue</code>
Message selection	By message type	By priority (highest first)
Notification	Not available	<code>mq_notify()</code> can signal when a message arrives
Compile	<code>gcc file.c</code>	<code>gcc file.c -lrt</code>

Short POSIX example (sender side):

```
#include <mqqueue.h>
#include <fcntl.h>
#include <string.h>
#include <stdio.h>

int main(void) {
    struct mq_attr attr = {0};
    attr.mq_maxmsg = 10;      // max messages in queue
    attr.mq_msgsize = 100;    // max size of each message

    mqd_t mq = mq_open("/myqueue", O_CREAT | O_WRONLY, 0666, &attr);
    if (mq == (mqd_t)-1) { perror("mq_open"); return 1; }

    const char *msg = "Hello POSIX queue";
```

```
mq_send(mq, msg, strlen(msg) + 1, 0); // priority 0

mq_close(mq);
return 0;
}
```

9. Useful Commands

Command	What it does
<code>ipcs -q</code>	List all System V message queues
<code>ipcs -q -i <id></code>	Show details of one queue
<code>ipcrm -q <msqid></code>	Remove a queue by ID
<code>ls /dev/mqueue</code>	List POSIX message queues (if mounted)

If your program crashes before calling `msgctl(IPC_RMID)`, the queue stays in the kernel. Use `ipcs -q` and `ipcrm` to clean up.

10. Advantages and Disadvantages

Advantages

- **Easy to use.** No manual memory sharing or locking.
- The kernel handles **synchronization**.
- Sender and receiver need **not run at the same time**, since messages wait in the queue.
- **Message types or priorities** let receivers choose what to read.
- Works for **many senders and many receivers**.

Disadvantages

- **Slower than shared memory**, because data is copied twice (user to kernel to user).
 - **Size limits**: each message and the total queue have maximum sizes (see `ipcs -l`).
 - Only works on the **same machine** (for distributed use, sockets are needed).
 - Queues can **persist** after the program ends if not removed.
 - Not suitable for very large data.
-

11. Real-World Use Cases

Use Case	Example
Task queues	A web server puts jobs (send an email, resize an image) in a queue, and worker processes pick them up
Logging systems	Many processes send log messages to one logger process
Operating system services	Communication between system daemons
Embedded and real-time systems	Priority-based messages between tasks
Message brokers (RabbitMQ, Kafka, ActiveMQ)	The same concept scaled to distributed systems

12. Summary

- In **message passing**, processes communicate through the kernel using `send` and `receive`.
- Design choices: **direct vs. indirect**, **blocking vs. non-blocking**, and **buffer capacity**.
- A **message queue** is a kernel-held list of typed messages. It uses indirect communication.
- System V uses `msgget`, `msgsnd`, `msgrcv`, `msgctl`. POSIX uses `mq_open`, `mq_send`, `mq_receive`, `mq_close`.
- Message passing is **simpler and safer** than shared memory, but **slower**.

13. References

1. Silberschatz, Galvin, Gagne. *Operating System Concepts* (Chapter 3: Interprocess Communication).
2. Linux manual pages: `man 2 msgget`, `man 2 msgsnd`, `man 2 msgrcv`, `man 2 msgctl`, `man 7 mq_overview`.
3. Kerrisk, M. *The Linux Programming Interface* (Chapters 46 and 52).
4. GeeksforGeeks: IPC using Message Queues.